

Introduction to C++ and Object Oriented Concepts

Mrs. Deepashree Mehendale

Procedure Oriented Programming Language

- In the procedure oriented approach, the problem is viewed as sequence of things to be done such as reading , calculation and printing.
- Procedure oriented programming basically consist of writing a list of instruction or actions for the computer to follow and organizing these instruction into groups known as functions.

Structure of Procedure Oriented Programming

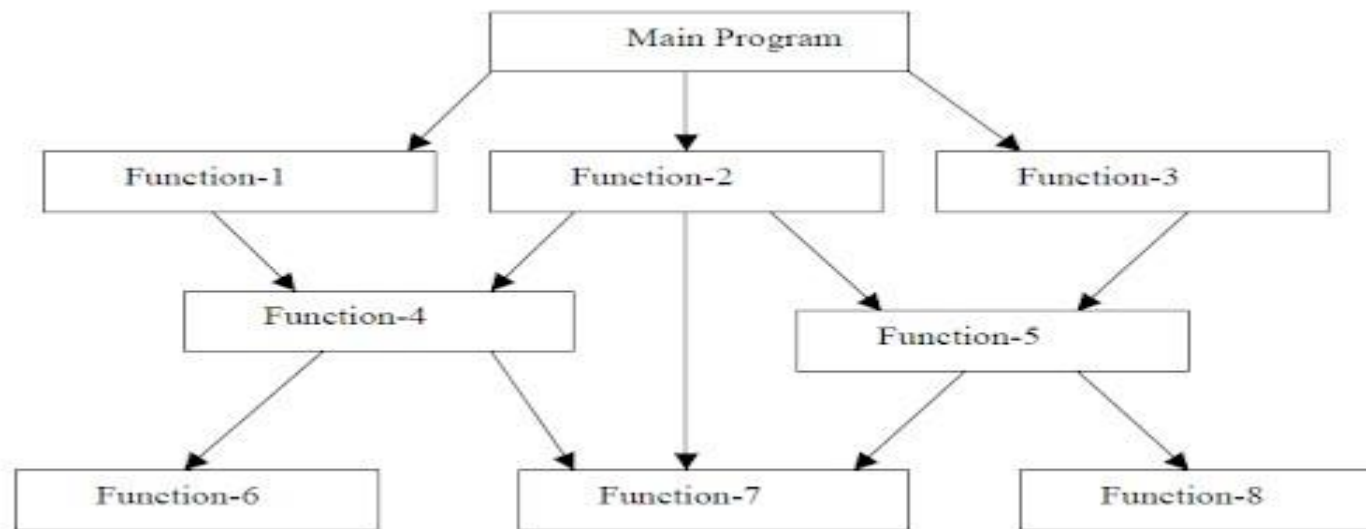


Fig. 1.2 Typical structure of procedural oriented programs

The disadvantage of the Procedure Oriented programming languages is:

1. Inability to reuse code.
2. It does not model real word problem very well
3. No data hiding

Characteristics of Procedure Oriented Programming:

1. Emphasis is on doing things(algorithm)
2. Large programs are divided into smaller programs known as functions.
3. Most of the functions share global data
4. Data move openly around the system from function to function
5. Function transforms data from one form to another.
6. Employs top-down approach in program design

Object Oriented Programming

- Object oriented programming is an approach that provides a way of modularizing programs by creating partitioned memory area for both data and functions that can be used as templates for creating copies of such modules on demand.

Features of the Object Oriented programming

1. Emphasis is on doing rather than procedure.
2. Programs are divided into what are known as objects.
3. Data structures are designed such that they characterize the objects.
4. Functions that operate on the data of an object are tied together in the data structure.
5. Data is hidden and can't be accessed by external functions.
6. Objects may communicate with each other through functions.
7. New data and functions can be easily added.
8. Follows bottom-up approach in program design.

Difference between POP and OOP

POP	OOP
Structure oriented.	Object oriented.
Program is divided into functions.	Program is divided into objects.
Top-down approach.	Bottom-up approach.
Inheritance is not allowed.	Inheritance property is used.
It doesn't use access specifier.	It uses access specifier.
No data hiding.	Encapsulation is used to hide the data.

BASIC CONCEPTS OF OBJECTS ORIENTED PROGRAMMING

1. Objects
2. Classes
3. Data abstraction and Encapsulation
4. Inheritance
5. Polymorphism
6. Dynamic binding
7. Message passing

OBJECTS

- Objects are the basic run-time entities in an object-oriented system. They may represent a person, a place, a bank account, a table of data or any item that the program must handle. The fundamental idea behind object oriented approach is to combine both data and function into a single unit and these units are called objects. The term objects means a combination of data and program that represent some real word entity. For example: consider an example named Amit; Amit is 25 years old and his salary is 2500. The Amit may be represented in a computer program as an object. The data part of the object would be (name: Amit, age: 25, salary: 2500)

CLASS

- A group of objects that share common properties for data part and some program part are collectively called as class.
- In C ++ a class is a new data type that contains member variables and member functions that operate on the variables.

DATA ABSTRACTION

- Abstraction refers to the act of representing essential features without including the back ground details or explanations. Classes use the concept of abstraction and are defined as size, width and cost and functions to operate on the attributes.

DATA ENCAPSALATION

- The wrapping up of data and function into a single unit (called class) is known as encapsulation. The data is not accessible to the outside world and only those functions which are wrapped in the class can access it. These functions provide the interface between the objects data and the program.

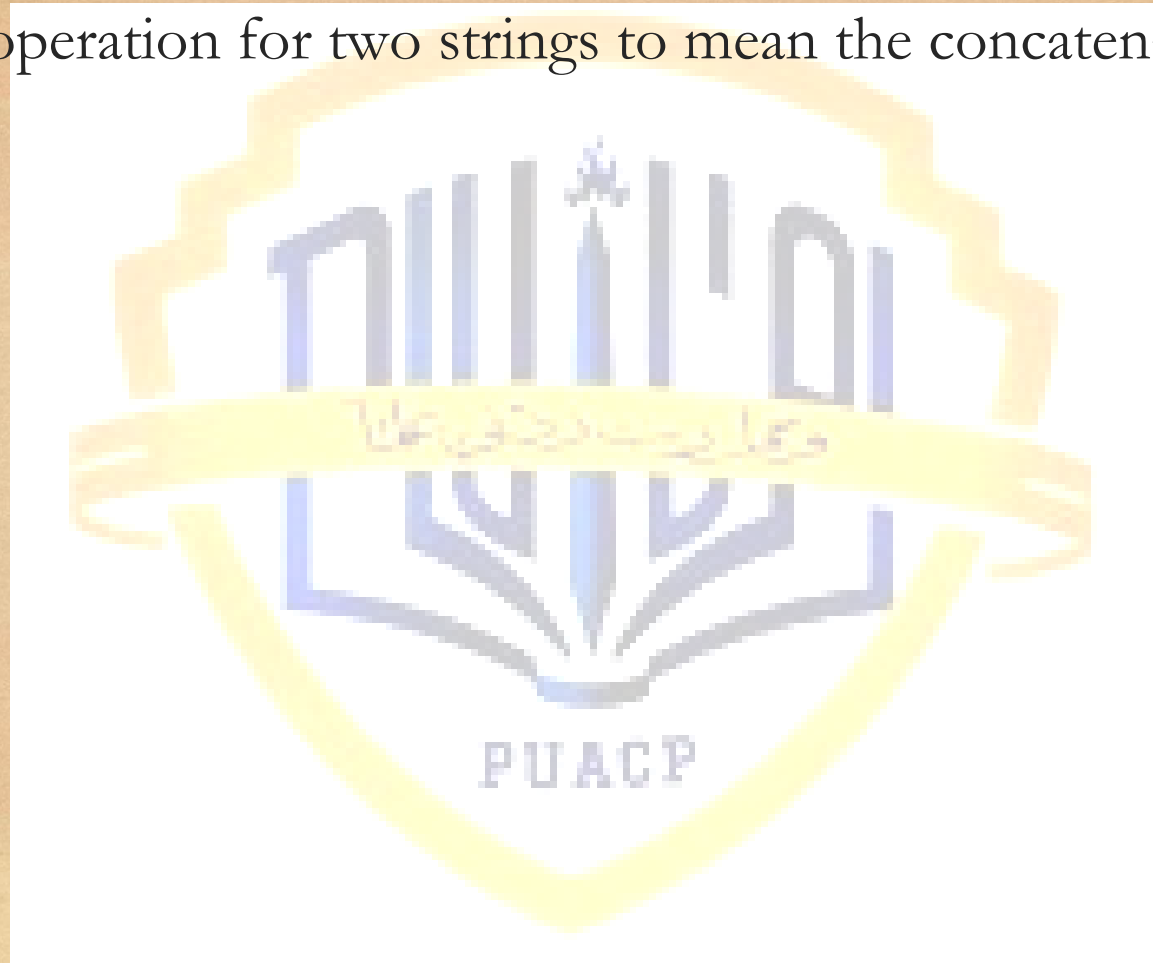
INHERITENCE :

- Inheritance is the process by which objects of one class acquire the properties of another class. The concept of inheritance provides the idea of reusability. This mean that we can add additional features to an existing class with out modifying it. This is possible by designing a new class which will have the combined features of both the classes.

POLYMORPHISM:

- Polymorphism means the ability to take more than one form. An operation may exhibit different instance. The behavior depends upon the type of data used in the operation. A language feature that allows a function or operator to be given more than one definition. The types of the arguments with which the function or operator is called determines which definition will be used. Overloading may be operator overloading or function overloading. It is able to express the operation of addition by a single operator say '+'. When this is possible you use the expression $x + y$ to denote the sum of x and y , for many different types of x and y ; integers, float and complex no. You can even

define the $+$ operation for two strings to mean the concatenation of the strings.



DYNAMIC BINDING

- Binding refers to the linking of a procedure call to the code to be executed in response to the call.
- Dynamic binding means the code associated with a given procedure call is not known until the time of the call at run-time.
- It is associated with a polymorphic reference depends upon the dynamic type of that reference.

MESSAGE PASSING

- An object oriented program consists of a set of objects that communicate with each other.
- A message for an object is a request for execution of a procedure and therefore will invoke a function (procedure) in the receiving object that generates the desired result. Message passing involves specifying the name of the object, the name of the function (message) and information to be sent.

APPLICATION OF OOP:

- Real – Time systems.
- Simulation and modeling
- Object oriented databases.
- Hypertext, hypermedia and Expertext.
- AI and expert systems.
- Neural networks and parallel programming.
- Decision support and office automation systems.
- CIM / CAM / CAD system.

What is C++?

- C ++ is an Object Oriented programming language,
- C ++ was developed by Bjarne Stroustrup at AT & T Bell lab, USA in early eighties. C ++ was developed from C and simula 67 language.
- C ++ was early called 'C with classes'.

Output Operator

- The statement `cout << "Hello, world"` displays the string within quotes on the screen.
- It is a predefined object that corresponds to the standard output stream. Stream just refers to a flow of data and the standard Output stream normally flows to the screen display
- The statement `cout <<` also called the 'put to' operator directs the information on its right to the object on its left.

Input Operator:

- The statement `cin >> number1;` is an input statement and causes the program to wait for the user to type in a number. The number keyed in is placed in the variable `number1`.
- The identifier `cin` is a predefined object in C++ that corresponds to the standard input stream. Here this stream represents the keyboard.
- The operator `>>` is known as get from operator. It extracts value from the keyboard and assigns it to the variable on its right.

Cascading Of I/O Operator:

- Multiple use of << or >> operator in a statement is called as Cascading of I/O Operators.
- `cout<<<sum<<<<sum<<"\n"<<<average<>number1>>number2;`

STRUCTURE OF C++ PROGRAM

- Include files
- Class declaration
- Class functions, definition
- Main function program

Simple Hello World program

```
#include<iostream.h>

int main()
{
    cout<<"Hello World";
    return 0;
}
```


Chapter 3:- Programming in C++

Mrs. Deepashree Mehendale

Data Types

- All variables use data-type during declaration to restrict the type of data to be stored.
- Therefore, we can say that data types are used to tell the variables the type of data it can store. Whenever a variable is defined in C++, the compiler allocates some memory for that variable based on the data-type with which it is declared. Every data type requires a different amount of memory.

Types

1. **Primitive:-** These are built in or primitive data types and can be used directly. Eg:- int, float, char etc.
2. **Derived:-** The data-types that are derived from the primitive or built-in datatypes are referred to as Derived Data Type. For eg:- Array, Function, Pointer
3. **Abstract:-** These data types are defined by user itself. Like, defining a class in C++ or a structure. C++. For eg:- class, structure, union

Type Conversion in C++

- **Implicit Type Conversion** Also known as 'automatic type conversion'. Done by the compiler on its own.
- Generally takes place when in an expression more than one data type is present. In such condition type conversion (type promotion) takes place to avoid lose of data.
- All the data types of the variables are upgraded to the data type of the variable with largest data type.

Explicit Type Conversion

- **Explicit Type Conversion:** This process is also called type casting and it is user-defined. Here the user can typecast the result to make it of a particular data type. It can be done by using Assignment Operator or by using casting operator.

Reference Variable

- A reference variable is an alias, that is, another name for an already existing variable. Once a reference is initialized with a variable, either the variable name or the reference name may be used.
- **Three major differences between references and pointers are –**
- You cannot have NULL references. You must always be able to assume that a reference is connected to a legitimate piece of storage.
- Once a reference is initialized to an object, it cannot be changed to refer to another object. Pointers can be pointed to another object at any time.
- A reference must be initialized when it is created. Pointers can be initialized at any time.

Syntax of Reference Variable

- We can declare reference variables as follows.

int & r = i;

- Read the & in these declarations as reference.
- Thus, "r is an integer reference initialized to i"

Example

```
#include <iostream.h>
```

```
int main () {
```

```
    int  i;
```

```
    double d;
```

```
    int &r = i;
```

```
    double &s = d;
```

```
    i = 5;
```

```
    cout << "Value of i : " << i << endl;
```

```
    cout << "Value of i reference : " << r << endl;
```

Example cont...

```
d = 11.7;
```

```
cout << "Value of d : " << d << endl;
```

```
cout << "Value of d reference : " << s << endl;
```

```
}
```

Class

- Everything in C++ is associated with classes and objects, along with its attributes and methods. For example: in real life, a car is an **object**. The car has **attributes**, such as weight and color, and **methods**, such as drive and brake.
- Attributes and methods are basically **variables** and **functions** that belongs to the class. These are often referred to as "class members".
- A class is a user-defined data type that we can use in our program, and it works as an object constructor, or a "blueprint" for creating objects.

Example of class

- A class contains attributes and methods defined in it. It is defined by using class keyword.
-

For example:-

```
class A
{
    int a;                //Attribute defined
    public:
        void accept();    // Method defined
};
```

C++ Access Specifiers

- There are 3 access specifiers used in C++:-
 - a. Public
 - b. Private
 - c. Protected

Access Specifiers

- **public** - members are accessible from outside the class
- **private** - members cannot be accessed (or viewed) from outside the class
- **protected** - members cannot be accessed from outside the class, however, they can be accessed in inherited classes. It is mostly used in Inheritance.

Example of access specifier

```
class A {  
    public:    // Public access specifier  
    int x;    // Public attribute  
    private: // Private access specifier  
    int y;    // Private attribute  
};  
int main() {  
    A a;  
    a.x = 10; // Allowed (public)  
    a.y = 50; // Not allowed (private)  
}
```

Defining Member Functions

- Member functions are the functions, which have their declaration inside the class definition and works on the data members of that class. The definition of member functions can be inside or outside the definition of class. If the member function is defined inside the class definition it can be defined directly, but if its defined outside the class, then we have to use the scope resolution operator along with class name along with function name.

Member function defined inside the class:-

```
class student
{
int rollno;
char name[20];    //Data Members
```


public:

void getdata()

//Member Function defined inside the class

{

cout<<"enter rollno";

cin>>rollno;

cout<<"enter name";

cin>>name;

}

};

Member function defined outside the class

In this case we must first declare the function inside the class and then define the function outside the class using scope resolution operator.

```
class student
{
    int rollno;
    char name[20];
    char branch[20];           //Data Members
```

public:

void getdata(); //Function declaration inside the class

void putdata(); //Function declaration inside the class

};

void student :: getdata() // Use of scope resolution operator

{

cout<<"enter rollno";

cin>>rollno;

cout<<"enter name";

cin>>name;


```
cout<<"enter branch";
```

```
cin>>branch;
```

```
}
```

Array of Objects:

- Like array of other user-defined data types, an array of type class can also be created. The array of type class contains the objects of the class as its individual elements. Thus, an array of a class type is also known as an array of objects. An array of objects is declared in the same way as an array of any built-in data type.

The syntax for declaring an array of objects is

```
class_name array_name[size];
```

Simple example illustrating use of array of objects.

```
#include<iostream.h>
```

```
class books
```

```
{
```

```
char title [30];
```

```
float price ;
```

```
public:
```

```
void getdata ( );
```

```
void putdata ( );
```

```
};
```


Example cont...

```
void books :: getdata ()  
{  
    cout<<"Title:";  
    cin>>title;  
    cout<<"Price:";  
    cin>>price;  
}
```

Example cont...

```
void books :: putdata ( )
```

```
{
```

```
    cout<<"Title:"<<title<< "\n";
```

```
    cout<<"Price:"<<price<< "\n";
```

```
}
```

```
int main ( )
```

```
{
```

```
books b[4] ;           // Array of books created
```

```
for(int i=0;i<4;i++)
{
    cout<<"Enter details of book "<<(i+1)<<"\n";
    b[i].getdata( );
}
for(int i=0;i<4;i++)
{
    cout<<"\nBook "<<(i+1)<<"\n";
    b[i].putdata( ) ;
}
}
```


Manipulators

- **Manipulators** are helping functions that can modify the input/output stream. It is used to change the data display.
- **Types of Manipulators**
- There are various types of manipulators:

Example of Manipulators

Manipulator	Purpose	Syntax
endl	endl manipulator is used to terminate a line and flushes the buffer.	endl
setw	setw manipulator sets the width of the field assigned for the output. The field width determines the minimum number of characters to be written in some output representations.	setw(no of characters) for eg:- setw(10) This will set the width of the field to 10 characters
setfill	This is used after setw manipulator. If a value does not entirely fill a field, then the character specified in the setfill argument of the manipulator is used for filling the fields.	setfill(character) for eg:- setfill(*) This will fill the field with *
setprecision	It sets the decimal precision for floating point values.	setprecision(number of digits) for eg:- setprecision(2); This will print the floating point output to 2 decimal places.

Manipulators

- **Manipulators without arguments:** The most important manipulators defined by the **iostream library** are provided below.
 - **endl:** It is defined in ostream. It is used to enter a new line and after entering a new line it flushes (i.e. it forces all the output written on the screen or in the file) the output stream.
 - **ws:** It is defined in istream and is used to ignore the whitespaces in the string sequence.

Manipulators with Arguments

- **Manipulators with Arguments:** Some of the manipulators are used with the argument like `setw (20)`, `setfill (“*)`, and many more.
- These all are defined in the header file. If we want to use these manipulators then we must include this header file in our program.
- For Example, you can use `setw` manipulators to set minimum width and `setfill` manipulator to fill the empty space with any character you want.

Manipulators with Arguments

- Some important manipulators in `<iomanip>` are:
- **setw (val):** It is used to set the field width in output operations.
- **setfill (c):** It is used to fill the character 'c' on output stream.
- **setprecision (val):** It sets val as the new value for the precision of floating-point values.

Example of setw

```
/*C++ program to demonstrate example of setw manipulator.*/  
#include <iostream.h>  
#include <iomanip.h>  
int main()  
{  
    char bname[3][30]={"C","C++","java"};  
    int price[3] = {200,300,400};
```


Example of setw

```
int i;

    cout << setw(30) << "Book Name" << setw(20) << "Price" << endl;
    for(i=0; i< 3; i++)
    {
        cout << setw(30) << bname[i] << setw(20) << price[i] << endl;
    }
    return 0;
}
```

Setfill Example

```
#include <iostream.h>
#include <iomanip.h>
int main ( ) {
    cout << setfill ('x') << setw (10);
    cout << 24 << endl;
    return 0;
}
```

Setprecision Example

```
#include <iomanip.h>
```

```
#include <iostream.h>
```

```
int main()
```

```
{
```

```
    // Initializing the decimal
```

```
    double num = 3.142857142857;
```

```
    cout << "Before setting the precision: \n"
```

```
        << num << endl;
```


Setprecision Example

```
// Using setprecision()
cout << "Setting the precision using"
    << " setprecision to 5: \n"
    << setprecision(5);
cout << num << endl;
```

Setprecision Example

```
// Using setprecision()
cout << "Setting the precision using
      "<<" setprecision to 9 : \n "
      << setprecision(9);
cout << num << endl;
return 0;
}
```


Functions in C++

Mrs. Deepashree K Mehendale

Definition

- A **function** is a group of statements that together perform a task. Every C++ program has at least one function, which is **main()**, and all the most trivial programs can define additional functions.
- You can divide up your code into separate functions. How you divide up your code among different functions is up to you, but logically the division usually is such that each function performs a specific task.

Function Declaration

- A function **declaration** tells the compiler about a function's name, return type, and parameters. A function **definition** provides the actual body of the function.
- The C++ standard library provides numerous built-in functions that your program can call. For example, function **strcat()** to concatenate two strings

Defining a Function

The general form of a C++ function definition is as follows –

```
return_type function_name( parameter list )  
{  
    body of the function  
}
```


Defining a Function

- **Return Type** – A function may return a value. The **return_type** is the data type of the value the function returns. Some functions perform the desired operations without returning a value. In this case, the return_type is the keyword **void**.
- **Function Name** – This is the actual name of the function. The function name and the parameter list together constitute the function signature.

Defining a Function

- **Parameters** – A parameter is like a placeholder. When a function is invoked, you pass a value to the parameter. This value is referred to as actual parameter or argument. The parameter list refers to the type, order, and number of the parameters of a function. Parameters are optional; that is, a function may contain no parameters.
- **Function Body** – The function body contains a collection of statements that define what the function does.

Example

```
int max(int num1, int num2)
{
    int result;
    if (num1 > num2)
        result = num1;
    else
        result = num2;
    return result;
}
```


Calling a Function

- While creating a C++ function, you give a definition of what the function has to do. To use a function, you will have to call or invoke that function
- When a program calls a function, program control is transferred to the called function. A called function performs defined task and when its return statement is executed or when its function-ending closing brace is reached, it returns program control back to the main program.
- To call a function, you simply need to pass the required parameters along with function name, and if function returns a value, then you can store returned value.

Cont...

- If a function is to use arguments, it must declare variables that accept the values of the arguments. These variables are called the **formal parameters** of the function.
- The formal parameters behave like other local variables inside the function and are created upon entry into the function and destroyed upon exit.

Cont...

- The **call by value** method of passing arguments to a function copies the actual value of an argument into the formal parameter of the function. In this case, changes made to the parameter inside the function have no effect on the argument.
- By default, C++ uses call by value to pass arguments. In general, this means that code within a function cannot alter the arguments used to call the function. Consider the function **swap()** definition as follows.

Example

```
void swap(int x, int y) {  
    int temp;  
    temp = x; /* save the value of x */  
    x = y;    /* put y into x */  
    y = temp; /* put x into y */  
}
```

```
#include <iostream.h>
```

```
void swap(int x, int y);
```

```
int main () {
```

```
    int a = 100;
```

```
    int b = 200;
```

```
    cout << "Before swap, value of a :" << a << endl;
```

```
    cout << "Before swap, value of b :" << b << endl;
```

```
    swap(a, b);
```

```
    cout << "After swap, value of a :" << a << endl;
```

```
    cout << "After swap, value of b :" << b << endl;
```

```
} Here the values are not swapped
```

Call by pointer / address

- The **call by pointer** method of passing arguments to a function copies the address of an argument into the formal parameter. Inside the function, the address is used to access the actual argument used in the call. This means that changes
- To pass the value by pointer, argument pointers are passed to the functions just like any other value. So accordingly you need to declare the function parameters as pointer types as in the following function **swap()**, which exchanges the values of the two integer variables pointed to by its arguments. made to the parameter affect the passed argument.

Example

```
void swap(int *x, int *y) {  
    int temp;  
    temp = *x; /* save the value at address x */  
    *x = *y; /* put y into x */  
    *y = temp; /* put x into y */  
    return;  
}
```

Call by reference

- The **call by reference** method of passing arguments to a function copies the reference of an argument into the formal parameter. Inside the function, the reference is used to access the actual argument used in the call. This means that changes made to the parameter affect the passed argument.
- To pass the value by reference, argument reference is passed to the functions just like any other value. So accordingly you need to declare the function parameters as reference types as in the following function **swap()**, which exchanges the values of the two integer variables pointed to by its arguments.

Example

```
void swap(int &x, int &y) {  
    int temp;  
    temp = x; /* save the value at address x */  
    x = y;    /* put y into x */  
    y = temp; /* put x into y */  
    return;  
}
```


Default Values

- A default argument is a value provided in a function declaration that is automatically assigned by the compiler if the caller of the function doesn't provide a value for the argument with a default value.

Default Values for Parameters

- When you define a function, you can specify a default value for each of the last parameters. This value will be used if the corresponding argument is left blank when calling to the function.
- This is done by using the assignment operator and assigning values for the arguments in the function definition. If a value for that parameter is not passed when the function is called, the default given value is used, but if a value is specified, this default value is ignored and the passed value is used instead. Consider the following example –

```
#include <iostream.h>
```

```
int sum(int a, int b = 20) {
```

```
    int result;
```

```
    result = a + b;
```

```
    return (result);
```

```
}
```

```
int main () {
```

```
    int a = 100;
```

```
    int b = 200;
```

```
    int result;
```


Example cont....

```
result = sum(a, b);  
    cout << "Total value is :" << result << endl;  
    result = sum(a);  
    cout << "Total value is :" << result << endl;  
}
```

Total value is :300

Total value is :120

- Default arguments are different from constant arguments as constant arguments can't be changed whereas default arguments can be overwritten if required.
 - Default arguments are overwritten when calling function provides values for them.
-
- During calling of function, arguments from calling function to called function are copied from left to right. Therefore, `sum(10, 15, 25)` will assign 10, 15 and 25 to x, y, and z.
 - Once default value is used for an argument in function definition, all subsequent arguments to it must have default value. It can also be stated as default arguments are assigned from right to left. For example, the following function definition is invalid as subsequent argument of default variable z is not default.

```
#include<iostream.h>
```

```
// A function with default arguments, it can be called with
```

```
// 2 arguments or 3 arguments or 4 arguments.
```

```
int sum(int x, int y, int z=0, int w=0)
```

```
{
```

```
    return (x + y + z + w);
```

```
}
```



```
int sum(int x, int y, float z=0, float w=0)
```

```
{
```

```
    return (x + y + z + w);
```

```
}
```

```
int main()
```

```
{
```

```
    cout << sum(10, 15) << endl;
```

```
    cout << sum(10, 15, 25) << endl;
```

```
    cout << sum(10, 15, 25, 30) << endl;
```

```
    return 0;
```

```
}
```

Why Inline functions in C++

- When the program executes the function call instruction the CPU stores the memory address of the instruction following the function call, copies the arguments of the function on the stack and finally transfers control to the specified function. The CPU then executes the function code, stores the function return value in a predefined memory location/register and returns control to the calling function. This can become overhead if the execution time of function is less than the switching time from the caller function to called function (callee).

Why Inline functions in C++

- For functions that are large and/or perform complex tasks, the overhead of the function call is usually insignificant compared to the amount of time the function takes to run. However, for small, commonly-used functions, the time needed to make the function call is often a lot more than the time needed to actually execute the function's code. This overhead occurs for small functions because execution time of small function is less than the switching time.

Why Inline functions in C++

- C++ provides an inline functions to reduce the function call overhead. Inline function is a function that is expanded in line when it is called. When the inline function is called whole code of the inline function gets inserted or substituted at the point of inline function call. This substitution is performed by the C++ compiler at compile time. Inline function may increase efficiency if it is small.

Syntax of Inline function

- The syntax for defining the function inline is:

```
inline return-type function-name(parameters)
```

```
{
```

```
    // function code
```

```
}
```

Important Points

- Remember, inlining is only a request to the compiler, not a command. Compiler can ignore the request for inlining. Compiler may not perform inlining in such circumstances like:
 - 1) If a function contains a loop. (for, while, do-while)
 - 2) If a function contains static variables.
 - 3) If a function is recursive.
 - 4) If a function return type is other than void, and the return statement doesn't exist in function body.
 - 5) If a function contains switch or goto statement.

Advantages of Inline Function

- 1) Function call overhead doesn't occur.
- 2) It also saves the overhead of push/pop variables on the stack when function is called.
- 3) It also saves overhead of a return call from a function.
- 4) When you inline a function, you may enable compiler to perform context specific optimization on the body of function. Such optimizations are not possible for normal function calls. Other optimizations can be obtained by considering the flows of calling context and the called context.
- 5) Inline function may be useful (if it is small) for embedded systems because inline can yield less code than the function call preamble and return.

Example of Inline Function

```
#include <iostream.h>
```

```
inline int cube(int s)
```

```
{  
    return s*s*s;
```

```
}
```

```
int main()
```

```
{  
    cout << "The cube of 3 is: " << cube(3) << " \n";
```

```
    return 0;
```

```
}
```

```
//Output: The cube of 3 is: 27
```

Static Data Member:-

- It is generally used to store value common to the whole class. The static data member differs from an ordinary data member in the following ways :
 - Only a single copy of the static data member is used by all the objects.
 - It can be used within the class but its lifetime is the whole program.
- For making a data member static, we require :
- Declare it within the class using keyword static.
- Define it outside the class using scope resolution operator.

Example

```
class student
```

```
{
```

```
static int count;           //declaration within class
```

```
-----
```

```
-----
```

```
};
```

```
int student :: count;       //definition outside class
```

The definition outside the class is a must. We can also initialize the static data member at the time of its definition as:

```
int student :: count = 0;
```

Static Member Function:-

- A static member function can access only the static members of a class. We can do so by putting the keyword static before the name of the function while declaring it.

```
class student
```

```
{
```

```
static int count;
```

```
-----
```

```
public :
```

```
-----
```

Example cont....

```
static void showcount (void)           //static member function
{
cout<<"count"<< count<<"\n";
}
};
```


Difference between static and member function

- In C++, a static member function differs from the other member functions in the following ways:
 - Only static members of the same class can be accessed by a static member function.
 - It is called by using the name of the class rather than an object as given below:
- `Name_of_the_class :: function_name`
- For example:- `student::showcount();`

Friend Function

- If a function is defined as a friend function in C++, then the protected and private data of a class can be accessed using the function.
- By using the keyword friend compiler knows the given function is a friend function.
- For accessing the data, the declaration of a friend function should be done inside the body of a class starting with the keyword friend.

Declaration of friend function

```
class class_name
{
    friend data_type function_name(argument/s); // syntax of friend function.
}
```


Characteristics of a Friend function:

- The function is not in the scope of the class to which it has been declared as a friend.
- It cannot be called using the object as it is not in the scope of that class.
- It can be invoked like a normal function without using the object.
- It cannot access the member names directly and has to use an object name and dot membership operator with the member name.
- It can be declared either in the private or the public part.

Example of friend function

```
#include<iostream.h>
```

```
class calculate
```

```
{
```

```
int x,y;
```

```
public:
```

```
void readdata()
```

```
{
```

```
    cout<<"\n enter two numbers";
```

```
    cin>>x>>y;
```

```
}
```

```
friend void add (calculate calc);    //declaration of friend function
```

```
};
```

```
void add (calculate calc)           //definition of friend function
```

```
{
```

```
cout<<"sum is"<<calc.x+calc.y;
```

```
}
```

```
main( )
```

```
{
```

```
calculate calc;
```

```
calc.readdata( );
```

```
add(calc);                          //Call to friend function
```

```
}
```


C++ Friend class

- A friend class can access both private and protected members of the class in which it has been declared as friend.
- **Forward Declaration** refers to the beforehand declaration of the syntax or signature of an identifier, variable, function, class, etc. prior to its usage (done later in the program).

Forward Declaration class A

```
// Forward Declaration class A
```

```
class A;
```

```
// Definition of class A
```

```
class A{
```

```
    // Body
```

```
};
```

Example of friend function as bridge between two classes

Creating friend function as a bridge between two classes:-A friend function can act as a bridge between two classes.

```
#include<iostream.h>

class B;                //forward declaration

class A
{
int x;
public:
void getdata(int p)
{
    x=p; }
}
```



```
void display()
{
cout<<"x="<<x<<endl;
}
```

```
friend int sum(A d, B e);
```

```
};
```

```
class B
```

```
{
```

```
int y;
```

```
public:
```

```
void getdata(int q)
```

```
{  y=q; }
```

```
void display()
{
cout<<"\ny="<<y<<endl;
}
friend int sum(A d, B e);
};
int sum(A d, B e)
{
return (A.x + B.y);
}
```

```
int main ( )  
{  
A d1;  
B e1;  
d1.getdata(10);  
e1.getdata(20);  
d1.display();  
e1.display();  
cout<<sum(d1,e1);  
}
```


Function Overloading

- An Overloaded Function may have:-
 - Different types of parameters
 - Different numbers of parameters
 - Different sequence of parameters.

Example

`void print( );`

`void print(int);`

`void print(float);`

`void print(int a, float b);`

`void print(int a, float b);`

`void print(float b, int a);`

Simple Example

```
#include<iostream.h>
```

```
int mul(int,int);
```

```
float mul(float, int);
```

```
int mul(int a,int b)
```

```
{
```

```
    return a*b;
```

```
}
```



```
float mul(double x, int y)
```

```
{
```

```
    return x*y;
```

```
}
```

```
int main( )
```

```
{
```

```
    int ans1 = mul(6,7);
```

```
    float ans2 = mul(0.2,3);
```

```
    cout << "Mul of 2 integer is : " <<ans1<< endl;
```

```
    cout <<"Mul of one int and one double value is is : " <<ans2<< endl;
```

```
}
```

Advantages of Overloading :

- The function can perform different operations and hence eliminates the use of different function names for the same kind of operations .
- Program becomes easy to understand.
- Easy maintainability of the code .
- Function overloading brings flexibility in C++ programs.

Constructors and Destructors

- **Constructor** is a special member function of a class that is executed whenever we create new objects of that class.
- A constructor will have exact same name as the class and it does not have any return type at all, not even void. Constructors can be very useful for setting initial values for certain member variables.

Constructor cont...

- A constructor is different from normal functions in following ways:
- Constructor has same name as the class itself
- Constructors don't have return type
- A constructor is automatically called when an object is created.
- If we do not specify a constructor, C++ compiler generates a default constructor for us (expects no parameters and has an empty body).

Default Constructor

- A constructor which has no argument is known as default constructor. It is invoked at the time of creating object.

```
#include <iostream.h>
```

```
class Employee
```

```
{
```

```
public:
```

```
    Employee()
```

```
{
```

```
    cout<<"Default Constructor Invoked"<<endl;
```

```
}
```

```
};
```

Default Constructor

```
int main(void)
{
    Employee e1; //creating an object of Employee
    return 0;
}
```

As soon as object e1 is created the default constructor is invoked.

Parameterized Constructor

- A constructor which has parameters is called parameterized constructor. It is used to provide different values to distinct objects.

```
#include <iostream.h>
```

```
// declare a class
```

```
class rectangle{
```

```
    int length;
```

```
    int breadth;
```

```
public:
```

```
    // create parameterized constructor
```

```
    rectangle(int l, int b) {
```

```
        // initialize private variables
```

```
        length = l;
```

```
        breadth=b;
```

```
    }
```

```
int calculateArea() {  
    return length * breadth;  
}
```

```
};
```

```
int main() {  
    // create object and initialize data members  
    rectangle r1(10, 8);  
    rectangle r2(8, 6);  
    cout << "Area of Rectangle 1: " << r1.calculateArea() << endl;  
    cout << "Area of : Rectangle 2" << r2.calculateArea() << endl;  
}
```


Copy Constructor

- The copy constructor in C++ is used to copy data of one object to another.
- The **copy constructor** is a constructor which creates an object by initializing it with an object of the same class, which has been created previously. The copy constructor is used to –
 - Initialize one object from another of the same type.
 - Copy an object to pass it as an argument to a function.
 - Copy an object to return it from a function.

The most common form of copy constructor is shown here –

```
classname (const classname &obj)
{
    // body of constructor
}
```

Here, obj is a reference to an object that is being used to initialize another object.

Dynamic Constructor

- It is a constructor which allows to allocate the memory to the objects at run time.

- Memory is allocated at run time using new operator.
- By using this **constructor**, we can **dynamically** initialize the objects.
- Dynamic constructor is used to allocate the memory to the objects at the run time.
- Memory is allocated at run time with the help of 'new' operator.
- By using this constructor, we can dynamically initialize the objects.

New Operator

- The **new operator** denotes a request for memory allocation on the Free Store. If sufficient memory is available, **new operator** initializes the memory and returns the address of the newly allocated and initialized memory to the pointer variable
- The following is the syntax of new operator.
- **pointer_variable = new datatype;**
- In the above syntax, pointer_variable is pointer variable to whom the memory address is returned and datatype is the build-in data type for the memory.

```
#include <iostream.h>
```

```
int main () {
```

```
    int *ptr1 = NULL;
```

```
    ptr1 = new int;
```

```
    float *ptr2 = new float(223.324);
```

```
    *ptr1 = 28;
```

```
    cout << "Value of pointer variable 1 : " << *ptr1 << endl;
```

```
    cout << "Value of pointer variable 2 : " << *ptr2 << endl;
```

```
    delete ptr1;
```

```
    delete ptr2;
```

```
}
```

Delete operator

- It is used to delete the memory previously allocated by new operator.
- It releases memory pointed by pointer variable.
- syntax is
- `delete pointername;`

new operator

- new operator is also used to allocate a block(an array) of memory of type *data-type*.
- pointer-variable = **new** data-type[size];
- where size(a variable) specifies the number of elements in an array.
- Example: `int *p = new int[10];`
- Dynamically allocates memory for 10 integers continuously of type int and returns pointer to the first element of the sequence, which is assigned to p(a pointer). p[0] refers to first element, p[1] refers to second element and so on.

delete

- To free the dynamically allocated array pointed by pointer-variable, use following form of *delete*:
- `delete[] pointer-variable;`

Destructors

- Destructors in C++ are member functions in a class that delete an object. They are called when the class object goes out of scope such as when the function ends, the program ends, a delete variable is called etc.
- Destructors are different from normal member functions as they don't take any argument and don't return anything. Also, destructors have the same name as their class and their name is preceded by a tilde(~).

Rules

1. Destructor name should be same as that of class name.
2. It should be preceded by tilde operator
3. It is declared in public section
4. Destructor does not take any arguments
5. Destructors are not overloaded.
6. Destructor never contains return value.

Rules

- A destructor function is called automatically when the object goes out of scope like:
- when the function ends
- when the program ends
- a block containing local variables ends
- a delete operator is called

```
#include<iostream.h>

class Demo
{   private: int num1, num2;
public: Demo(int n1, int n2) {
cout<<"Inside Constructor"<<endl;
num1 = n1;
num2 = n2;
}

void display()
{   cout<<"num1 = "<< num1 <<endl;
    cout<<"num2 = "<< num2 <<endl; }
```


Operator Overloading

Reason

- By using operator overloading we can use +operator to add two user defined data type
- $c=a+b$; is allowed where all are built in data types
- But if you write $\text{obj3}=\text{obj1}+\text{obj2}$; directly is not allowed but allowed with Concept of Operator Overloading.

Introduction

- C++ allows you to specify more than one definition for a **function** name or an **operator** in the same scope, which is called **function overloading** and **operator overloading** respectively.
- An overloaded declaration is a declaration that is declared with the same name as a previously declared declaration in the same scope, except that both declarations have different arguments and obviously different definition (implementation).

Introduction cont..

- When you call an overloaded **function** or **operator**, the compiler determines the most appropriate definition to use, by comparing the argument types you have used to call the function or operator with the parameter types specified in the definitions. The process of selecting the most appropriate overloaded function or operator is called **overload resolution**.

Introduction cont..

- Operator overloading is a compile-time polymorphism in which the operator is overloaded to provide the special meaning to the user-defined data type. Operator overloading is used to overload or redefines most of the operators available in C++. It is used to perform the operation on the user-defined data type. For example, C++ provides the ability to add the variables of the user-defined data type that is applied to the built-in data types.
- The advantage of Operators overloading is to perform different operations on the same operand.

Operator that cannot be overloaded are as follows:

- **Operator that cannot be overloaded are as follows:**
- Scope operator (::)
- Sizeof
- member selector(.)
- member pointer selector(*)
- ternary operator(?:)

Syntax of Operator Overloading

```
return_type class_name :: operator op(argument_list)
```

```
{
```

```
    // body of the function.
```

```
}
```

- Where the **return type** is the type of value returned by the function.
- **class_name** is the name of the class.
- **operator op** is an operator function where op is the operator being overloaded, and the operator is the keyword

Rules for Operator Overloading

- Existing operators can only be overloaded, but the new operators cannot be overloaded.
- ~~The overloaded operator contains atleast one operand of the user-defined data type.~~
- We cannot use friend function to overload certain operators. However, the member function can be used to overload those operators.
- When unary operators are overloaded through a member function take no explicit arguments, but, if they are overloaded by a friend function, takes one argument.
- When binary operators are overloaded through a member function takes one explicit argument, and if they are overloaded through a friend function takes two explicit arguments.

Overloading Unary Operator using member function

```
#include<iostream.h>
```

```
class Number
```

```
{
```

```
    private:
```

```
        int x;
```

```
    public:
```

```
        Number(int p)
```

```
        {
```

```
            x = p;
```

```
        }
```



```
void operator -()
```

```
{  x = -x; }
```

```
void display()
```

```
{  cout<<"x = "<<x; }
```

```
};
```

```
int main()
```

```
{
```

```
    Number n(10);
```

```
    -n;
```

```
    n.display();
```

```
    return 0;
```

```
}
```

Overloading Unary Operator using friend function

```
#include<iostream.h>
```

```
class Number
```

```
{
```

```
int a=10;
```

```
int b=20;
```

```
int c=30;
```

```
public:
```

```
void getvalues()
```

```
{
```

```
cout<<"Values of A, B & C\n";
```

```
cout<<a<<"\n"<<b<<"\n"<<c<<"\n"<<endl;
```

```
}
```

```
void show()
{
    cout<<a<<"\n"<<b<<"\n"<<c<<"\n"<<endl;
}


---


void friend operator-(Number &x);    //Pass by reference
};
void operator-(Number &x)
{
    x.a = -x.a;    //Object name must be used as it is a friend function
    x.b = -x.b;
    x.c = -x.c;
}
```



```
int main()  
{  
    Number x1;  
    x1.getvalues();  
    cout<<"Before Overloading\n";  
    x1.show();  
    cout<<"After Overloading \n";  
    -x1;  
    x1.show();  
}
```

Type Conversion

Overloading typecasts

- C++ already knows how to convert between the built-in data types. However, it does not know how to convert any of our user-defined classes. That's where overloading of the typecast operators comes into play.
- **User-defined conversions** allow us to convert our class into another data type.

Type conversions in C++

- Type conversion is the process of converting one type of expression or item into another type. Type conversion can be implicit or explicit.
- Implicit / Automatic type conversion Implicit type conversion is applied when data items of different types are intermixed in an expression.
- This type of conversion is performed by the compiler itself so that no loss of data or information takes place. C++ compiler converts all items of expression into the largest data type present into the expression.

Type conversions in C++

- For example, if an expression contains items of the types like float, double and long double, compiler will implicitly convert all types to long double. Unsigned long integer is the longest integer data type, hence if one item in the expression is of the type unsigned long integer, then all int and short data type items will get implicitly converted into long integer data type.

Explicit type conversion / type casting

- Explicit type conversion is also known as type casting and it is implemented by using following syntax (data_type) expression;
- For example if there are variables like
- `int m = 35; int n = 6;`
- `float x; x = m / n; //here`
- `m = 35, n = 6` so `x = 5.0`
- `x = (float) m / (float) n // here m = 35.0, n=6.0` so `x = 5.8333`

Types of Conversion

- Built in data type to class type
- Class type to basic type
- One class type to another class type

Conversion between basic and user defined data types

- The conversions discussed above do not require special code, as only built in data types are used for conversion. Special programming is needed for conversion between user defined data items and basic data type, because compiler need to understand the structure and memory requirement of user defined data type before conversion. Different types of conversion are
 - Basic to user defined data type (class)
 - User defined (class) to basic data type
 - One user defined (class1) to another user defined data type (class2)

Conversion from basic data type to class type

- Conversion from basic data type to class type can be performed by using a single parameter in a constructor.
- The example below will explain it in detail

// program to convert quantity of weight
into kilograms and grams

```
#include<iostream.h>
```

```
class weight
```

```
{
```

```
    int kg, gm;
```

```
public: weight ()
```

```
{
```

```
    kg = gm = 0;
```

```
}
```

weight (int w) // this constructor will convert an integer to class type

{

kg = w / 1000;

gm = w % 1000;

}

void display ()

{

cout<<< kg<< gm;

}

```
int main ( )
```

```
{
```

```
    int wgm;
```

```
    cout<<"Enter weight in grams";
```

```
    cin>> wgm;
```

```
    weight sugar ;
```

```
    sugar = wgm; // wgm gets converted into kg and gm
```

```
    sugar.display ( );
```

```
    return 0;
```

```
}
```


Conversion from class type to basic type

- C++ has the option of overloading casting operator which can be used for conversion from class to basic type. The function used to overload casting operator is usually called conversion function.

Syntax is as follows:-

```
operator type_name (  
{ ..... function body ..... }
```

Rules

- It must be a class member
- It must not have a return type but a return statement
- Conversion function cannot accept any argument
- Being a member function, it is invoked by an object and values of that object are used inside the function. Hence no argument is passed to conversion function

One class type to another class type

- Conversions from one class to another class type can be carried out either by a constructor or by a conversion function. If objA is object of class A and objB is the object of class B. To convert class B data type to class A data type following statement can help `objA = objB` ; For this conversion, class B is known as source class and class A is known as destination class.
- When a class is to be converted, casting operator function can be used in source class. `operator type_name ( )`

One class type to another class type

- To convert source class type data item to destination class type data item, above function should be a member function of the source class and type_name should be the type of destination class.

Inheritance

Mrs. Deepashree K. Mehendale

Definition

- One of the most important concepts in object-oriented programming is that of inheritance. **Inheritance** allows us to define a class in terms of another class, which makes it easier to create and maintain an application. This also provides an opportunity to **reuse** the code functionality and fast implementation time.
- When creating a class, instead of writing completely new data members and member functions, the programmer can designate that the new class should inherit the members of an existing class.
- This existing class is called the **base** class, and the new class is referred to as the **derived** class.

Base and Derived Classes

- In C++, inheritance is a process in which one object acquires all the properties and behaviors of its parent object automatically. In such way, you can reuse, extend or modify the attributes and behaviors which are defined in other class.
- In C++, the class **which inherits** the members of another class is called **derived class** and the class **whose members are inherited** is called **base class**. The derived class is the specialized class for the base class.

Advantage of C++ Inheritance

- **Code reusability:** Now you can reuse the members of your parent class. So, there is no need to define the member again. So less code is required in the class.

Types Of Inheritance

- Single inheritance
- Multiple inheritance
- Multilevel inheritance
- Hierarchical inheritance
- Hybrid inheritance

Types Of Inheritance

- When deriving a class from a base class, the base class may be inherited through **public**, **protected** or **private** inheritance. The type of inheritance is specified by the **access specifier** as explained above.
- We hardly use **protected** or **private** inheritance, but **public** inheritance is commonly used. While using different type of inheritance, following rules are applied –

- **Public Inheritance** – When deriving a class from a **public** base class, **public** members of the base class become **public** members of the derived class and **protected** members of the base class become **protected** members of the derived class. A base class's **private** members are never accessible directly from a derived class, but can be accessed through calls to the **public** and **protected** members of the base class.
- **Protected Inheritance** – When deriving from a **protected** base class, **public** and **protected** members of the base class become **protected** members of the derived class.
- **Private Inheritance** – When deriving from a **private** base class, **public** and **protected** members of the base class become **private** members of the derived class.

Rules

Base class member access specifier	Type of Inheritance		
	Public	Protected	Private
Public	Public	Protected	Private
Protected	Protected	Protected	Private
Private	Not accessible (Hidden)	Not accessible (Hidden)	Not accessible (Hidden)

Syntax of Derived Class

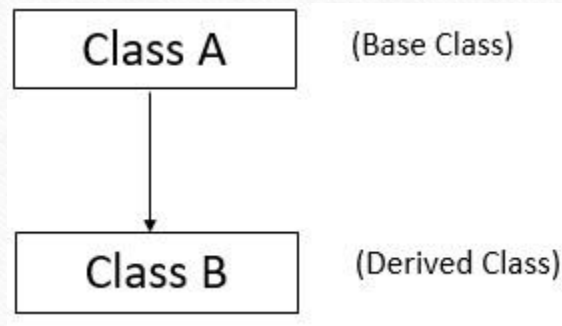
```
class derived_class_name : visibility-mode base_class_name
{
    // body of the derived class.
}
```

Syntax of Derived Class

- **Where,**
- **derived_class_name:** It is the name of the derived class.
- **visibility mode:** The visibility mode specifies whether the features of the base class are publicly inherited or privately inherited. It can be public or private.
- **base_class_name:** It is the name of the base class.

C++ Single Inheritance

- **Single inheritance** is defined as the inheritance in which a derived class is inherited from the only one base class.



Example of Single Inheritance

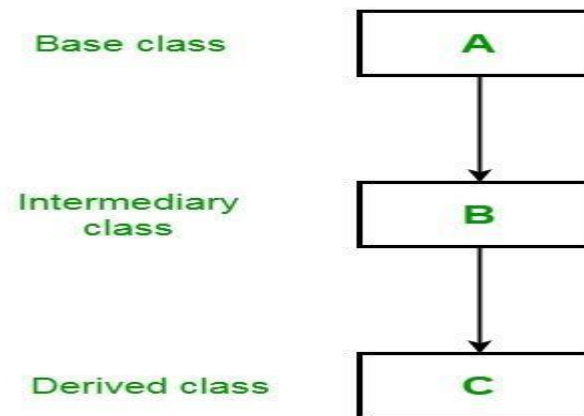
```
#include <iostream.h>

class Account
{
    public:
        float salary = 60000;
};
```

```
class Programmer: public Account {  
    public:  
    float bonus = 5000;  
};  
  
int main()  
{  
    Programmer p1;  
    cout<<"Salary: "<<p1.salary<<endl;  
    cout<<"Bonus: "<<p1.bonus<<endl;  
}
```

Multilevel Inheritance

- In multilevel inheritance a class *A* serves as a base class for the derived class *B* and class *C* serves as a derived class for class *B*.
- Here class *B* is called as **intermediate base class** because it provides a link between class *A* and class *C*



Simple Example of Multilevel Inheritance

```
#include <iostream.h>

class A {
    public:
        void display() {
            cout<<"Base class content.";
        }
};
```

```
class B : public A {};
```

```
class C : public B {};
```

```
int main() {
```

```
    C obj;
```

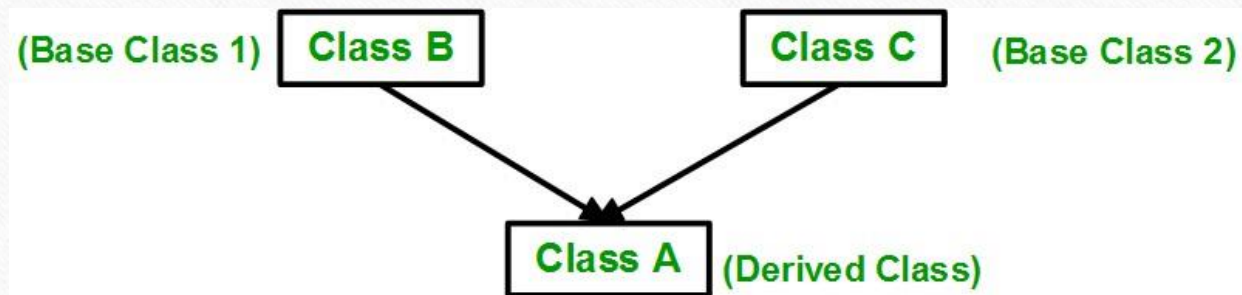
```
    obj.display();
```

```
    return 0;
```

```
}
```

Multiple Inheritance

- **Multiple Inheritance** is a feature of **C++** where a class can **inherit** from more than one classes.



Simple Example of Multiple Inheritance

```
#include <iostream.h>
```

```
class A
```

```
{
```

```
    public:
```

```
    int a = 5;
```

```
    A()
```

```
{
```

```
    cout << "Constructor for class A" << endl;
```

```
}
```

```
};
```

```
class B
```

```
{
```

```
    public:
```

```
    int b = 10;
```

```
    B()
```

```
{
```

```
    cout << "Constructor for class B" << endl;
```

```
}
```

```
};
```

```
class C: public A, public B
```

```
{
```

```
    public:
```

```
    int c = 20;
```

```
    C()
```

```
{
```

```
    cout << "Constructor for class C" << endl;
```

```
    cout<<"Class C inherits from class A and class B" << endl;
```

```
}
```

```
};
```

```
int main() {
```

```
    C obj;
```

```
    cout<<"a = "<< obj.a <<endl;
```

```
    cout<<"b = "<< obj.b <<endl;
```

```
    cout<<"c = "<< obj.c <<endl;
```

```
}
```

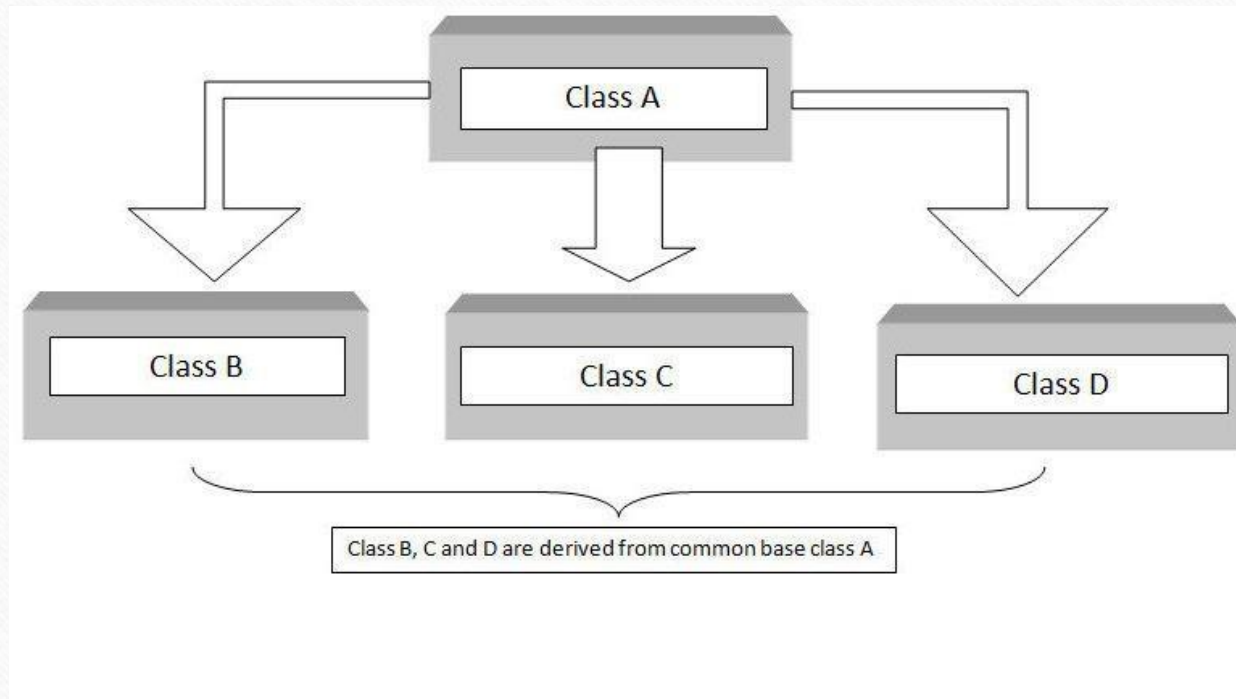
Hierarchical inheritance.

- When several classes are derived from common base class it is called **hierarchical inheritance**.
- In **C++ hierarchical inheritance**, the feature of the base class is inherited onto more than one sub-class.

Example of Hierarchical inheritance.

- **For example** consider a base class Account and derived 2 classes from Account ie:- Saving Account and Current Account from base class.
- In Saving and Current Account we add one data member balance and check the balance according to the type of Account.

Hierarchical Inheritance



Example

```
#include <iostream.h>
class A //single base class
{
    public:
    int x, y;
    void getdata()
    {
        cout << "\nEnter value of x and y: \n"; cin >> x >> y;
    }
};
```

Example cont...

```
class B : public A      //B is derived from class base
{
    public:
    void product()
    {
        cout << "\nProduct= " << x * y;
    }
};
```


Example cont..

```
class C : public A //C is also derived from class base
{
    public:
    void sum()
    {
        cout << "\nSum= " << x + y;
    }
};
```

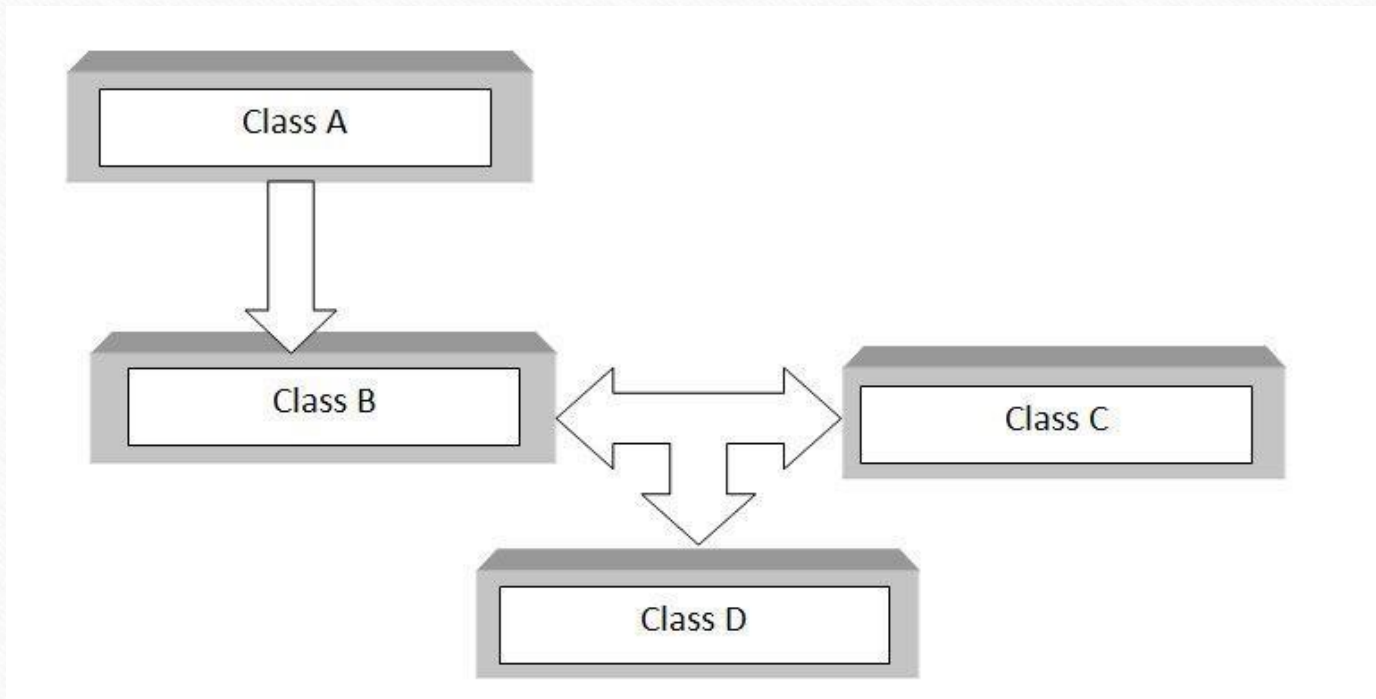
Example cont..

```
int main()
{
    B obj1;    //object of derived class B
    C obj2;    //object of derived class C
    obj1.getdata();
    obj1.product();
    obj2.getdata();
    obj2.sum();
    return 0;
}
```

Hybrid Inheritance

- The inheritance in which the derivation of a class involves more than one form of any inheritance is called **hybrid inheritance**.
- Basically **C++ hybrid inheritance** is combination of two or more types of inheritance. It can also be called multipath inheritance.

Hybrid Inheritance



Hybrid Inheritance

- Above block diagram shows the hybrid combination of **single inheritance and multiple inheritance**. Hybrid inheritance is used in a situation where we need to apply more than one inheritance in a program.
- As in other inheritance, based on the visibility mode used or access specifier used while deriving, the properties of the base class are derived. Access specifier can be private, protected or public.

```
#include <iostream.h>
```

```
class A
```

```
{
```

```
    public:
```

```
    int x;
```

```
};
```

```
class B : public A
```

```
{
```

```
    public:
```

```
    B()    //constructor to initialize x in base class A
```

```
    {
```

```
        x = 10;
```

```
    }
```

```
};
```



```
class C {  
public:  
    int y;  
    C() //constructor to initialize y  
    {  
        y = 4;  
    }  
};  
class D : public B, public C //D is derived from class B and class C  
{  
    public:  
    void sum()  
    {  
        cout << "Sum= " << x + y;  
    }  
};
```

Example cont..

```
int main()
{
    D obj1;      //object of derived class D
    obj1.sum();
    return 0;
}               //end of program
```

Function Overriding

- As we know, Inheritance is a feature of OOP that allows us to create derived classes from a base class. The derived classes inherit features of the base class.
- Suppose, the **same function is defined** in both the derived class and the based class. Now if we call this function using the object of the derived class, the function of the derived class is executed.
- This is known as **function overriding** in C++. The function in derived class overrides the function in base class.


```
#include <iostream.h>
```

```
class Base
```

```
{
```

```
    public: void print( )
```

```
{
```

```
    cout << "Base Function" << endl;
```

```
}
```

```
};
```

```
class Derived : public Base
{
    public: void print( )
    {
        cout << "Derived Function" << endl;
    }
};

int main( )
{
    Derived derived1;
    derived1.print( );
}
```

// C++ program to access overridden function // in main() using the scope resolution operator ::

```
#include <iostream.h>
```

```
class Base
```

```
{
```

```
    public: void print()
```

```
    {
```

```
        cout << "Base Function" << endl;
```

```
    }
```

```
};
```



```
class Derived : public Base
{
    public: void print()
    {
        cout << "Derived Function" << endl;
    }
};

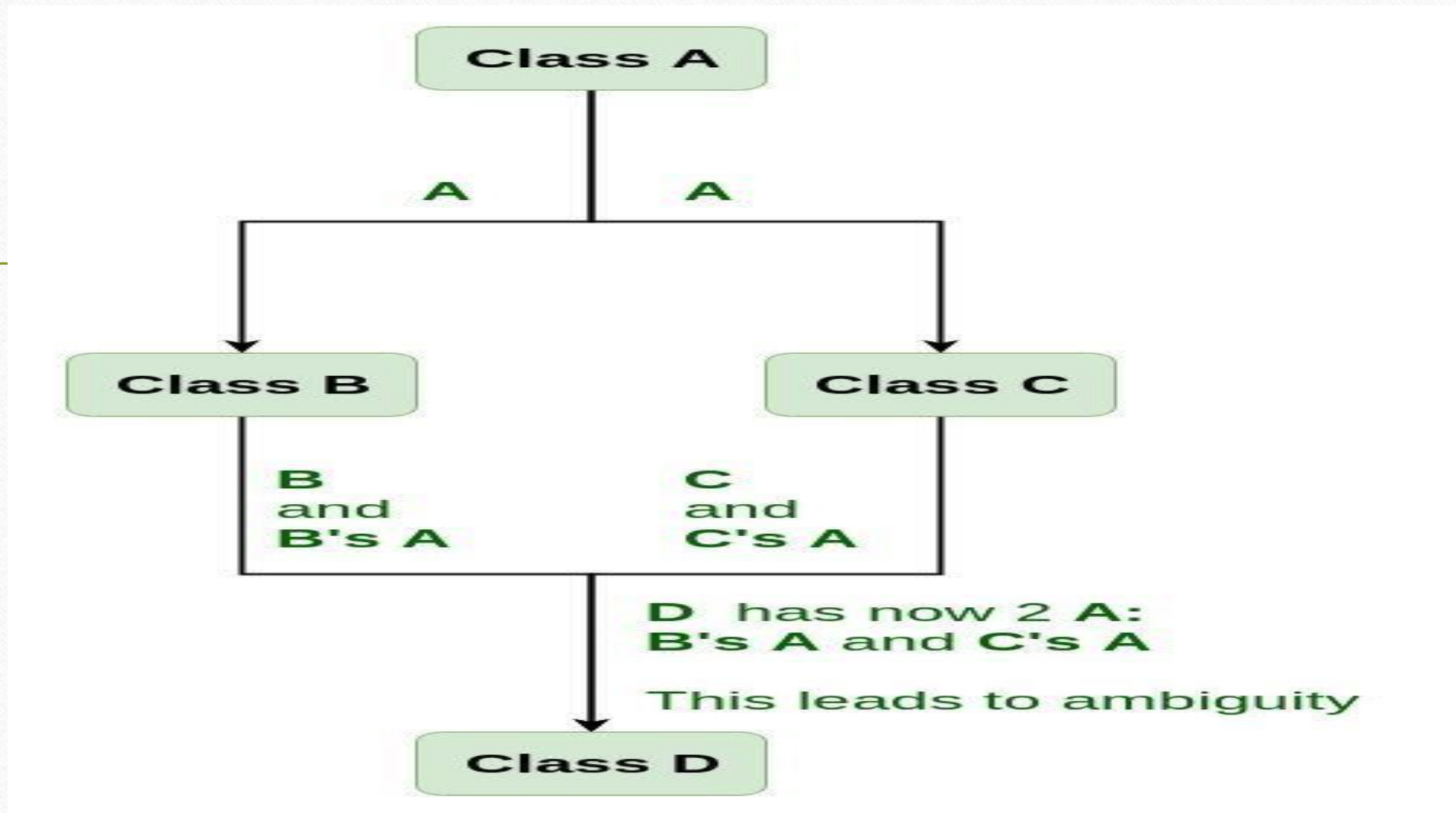
int main()
{
    Derived derived1, derived2;
    derived1.print(); // access print() function of the Base class
    derived2.Base::print();
}
```

Virtual Base Classes:

- Virtual base classes are used in virtual inheritance in a way of preventing multiple “instances” of a given class appearing in an inheritance hierarchy when using multiple inheritances.

Need for Virtual Base Classes:

- Consider the situation where we have one class **A** .
- This class **A** is inherited by two other classes **B** and **C**. Both these class are inherited into another in a new class **D** as shown in figure below.



Virtual Base Classes:

- As we can see from the figure that data members/function of class **A** are inherited twice to class **D**. One through class **B** and second through class **C**. When any data / function member of class **A** is accessed by an object of class **D**, ambiguity arises as to which data/function member would be called? One inherited through **B** or the other inherited through **C**. This confuses compiler and it displays error.

Syntax of Virtual Base Class

- We need to use the keyword **virtual** while deriving the class so as to remove the ambiguity.
- For Example:-

```
class A
{ ....}

class B: public virtual A
{ ..... }

class C: virtual public A
{ ..... }
```


Example of Virtual Base class

```
#include<iostream.h>

class student {
    int rno;
public:
    void getnumber() {
        cout << "Enter Roll No:";
        cin>>rno;
    }
    void putnumber() {
        cout << "\n\n\tRoll No:" << rno << "\n";
    }
};
```

```
class test : virtual public student {  
public:  
    int part1, part2;  
    void getmarks() {  
        cout << "Enter Marks\n";  
        cout << "Part1:";  
        cin >> part1;  
        cout << "Part2:";  
        cin >> part2;  
    }  
    void putmarks () {  
        cout << "\tMarks Obtained\n";  
        cout << "\n\tPart1:" << part1 << "\n\tPart2:" << part2;  
    }  
};
```

```
class sports : public virtual student {  
public:  
    int score;  
    void getscore() {  
        cout << "Enter Sports Score:";  
        cin >> score;  
    }  
    void putscore() {  
        cout << "\n\tSports Score is:" << score;  
    }  
};  
class result : public test, public sports {  
    int total;  
public:
```



```
void display()
{
    total = part1 + part2 + score;
    putnumber();
    putmarks();
    putscore();
    cout << "\n\tTotal Score:" << total;
}
};
```

```
void main( ) {
    result obj;
    obj.getnumber();
    obj.getmarks();
    obj.getscore();
    obj.display();
}
```

C++ Virtual Function

- A C++ **virtual function** is a member function in the base class that you redefine in a derived class. It is declared using the virtual keyword.
- It is used to tell the compiler to perform dynamic linkage or late binding on the function.
- There is a necessity to use the single pointer to refer to all the objects of the different classes. So, we create the pointer to the base class that refers to all the derived objects. But, when base class pointer contains the address of the derived class object, always executes the base class function. This issue can only be resolved by using the 'virtual' function.
- A 'virtual' is a keyword preceding the normal declaration of a function.
- When the function is made virtual, C++ determines which function is to be invoked at the runtime based on the type of the object pointed by the base class pointer.

Late binding or Dynamic linkage

- In late binding function call is resolved during runtime. Therefore compiler determines the type of object at runtime, and then binds the function call.

- A C++ virtual function is a member function in the base class that you redefine in a derived class. It is declared using the virtual keyword.
- It is used to tell the compiler to perform dynamic linkage or late binding on the function.

- There is a necessity to use the single pointer to refer to all the objects of the different classes. So, we create the pointer to the base class that refers to all the derived objects. But, when base class pointer contains the address of the derived class object, always executes the base class function. This issue can only be resolved by using the 'virtual' function.
- A 'virtual' is a keyword preceding the normal declaration of a function.
- When the function is made virtual, C++ determines which function is to be invoked at the runtime based on the type of the object pointed by the base class pointer.

Rules of Virtual Function

- Virtual functions must be members of some class.
- Virtual functions cannot be static members.
- They are accessed through object pointers.
- They can be a friend of another class.
- A virtual function must be defined in the base class, even though it is not used.
- The prototypes of a virtual function of the base class and all the derived classes must be identical. If the two functions with the same name but different prototypes, C++ will consider them as the overloaded functions.
- We cannot have a virtual constructor, but we can have a virtual destructor
- Consider the situation when we don't use the virtual keyword.

Pure Virtual Function

- A virtual function is not used for performing any task. It only serves as a placeholder.
- When the function has no definition, such function is known as "**do-nothing**" function.
- The "**do-nothing**" function is known as a **pure virtual function**. A pure virtual function is a function declared in the base class that has no definition relative to the base class.
- A class containing the pure virtual function cannot be used to declare the objects of its own, such classes are known as abstract base classes.
- The main objective of the base class is to provide the traits to the derived classes and to create the base pointer used for achieving the runtime polymorphism.

Pure Virtual Function

- **Pure virtual function can be defined as follows:**

virtual void display() = 0;

Abstract Class:- C++ abstract class is conceptually a class that cannot be instantiated and it should be implemented as a class that has one or more pure virtual (abstract) functions. A pure virtual function is one which must be overridden by any concrete (i.e., non-abstract) derived class. We cannot create the instance of an abstract class. We can create pointer and reference of base abstract class points to the instance of the child class.

Example

```
class Base
```

```
{
```

```
    public: virtual void show() = 0; // Pure Virtual Function
```

```
};
```

Example

```
class derived: public Base
{
public: void show()
{
    cout << "Implementation of Virtual Function in Derived class \n";
}
};
```

```
int main()
{
Base obj; //Compile Time Error
Base *b;
Derived d;
b = &d;
b->show();
}
```

File Handling and Exception Handling

Mrs. Deepashree K Mehendale

Why do we need a File

- Files are used to store data in a storage device permanently.
- File handling provides a mechanism to store the output of a program in a file and to perform various operations on it.
- So far, we have been using the **iostream** standard library, which provides **cin** and **cout** methods for reading from standard input and writing to standard output respectively.

Why do we need a File

- Variables are usually stored in RAM. This is either on the Heap (e.g. global variables, static variables in methods/functions) or on the Stack (e.g. non-static variables declared within a method/function).
- Stack and Heap are both RAM.

For Example

```
class account
```

```
{
```

```
    int accno;
```

```
    char accname[20];
```

```
    float balance;
```

```
public:
```

```
    void getdata( );
```

```
};
```

```
void account::getdata()  
{  
    cout<<"enter acc no, name and balance";  
    cin>>accno>>accname>>bal; // Data is stored in a variable  
}  
main()  
{  
    account a;  
    a.getdata();  
}
```

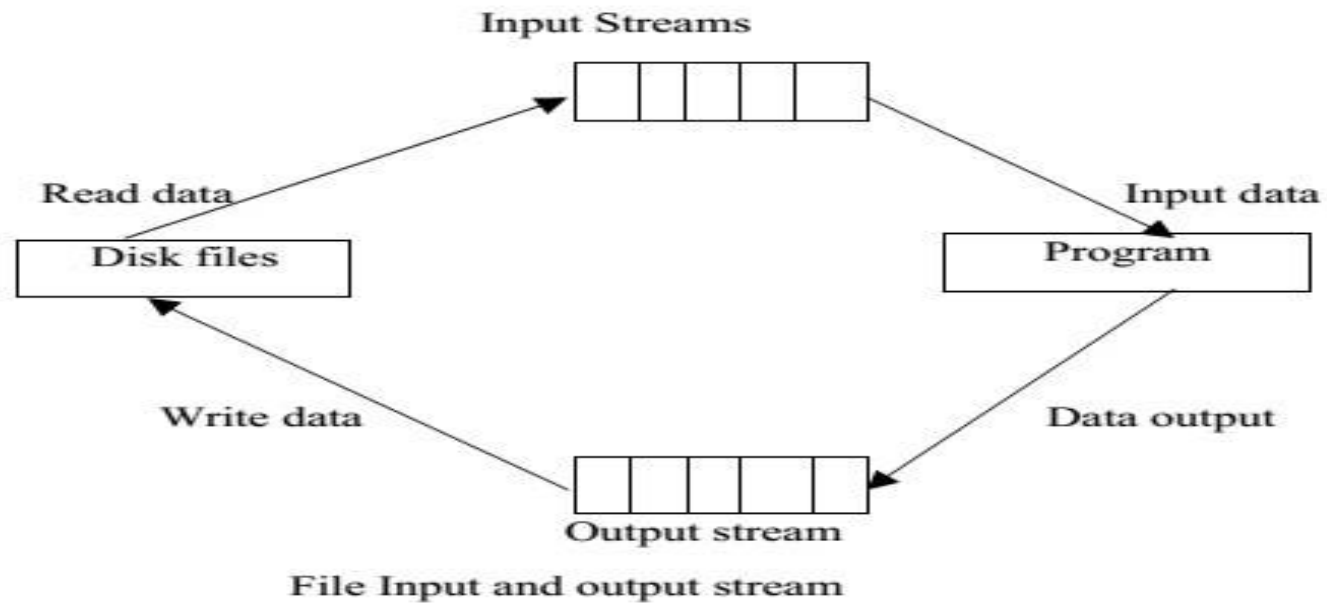

What is a File

- Many real-life scenarios are there that handle a large number of data, and in such situations, you need to use some secondary storage to store the data. The data are stored in the secondary device using the concept of files.
- **Files** are the collection of related data stored in a particular area on the disk. Programs can be written to perform read and write operations on these files.

What is a File

- Files are either text file or binary file.
- A text file can be created easily, easily read or printed
- Binary files are data files they cannot be easily read or write but the size of binary file is smaller than that of text file.
- Also the speed at which programs can read data from binary file is very high as compared to text file.

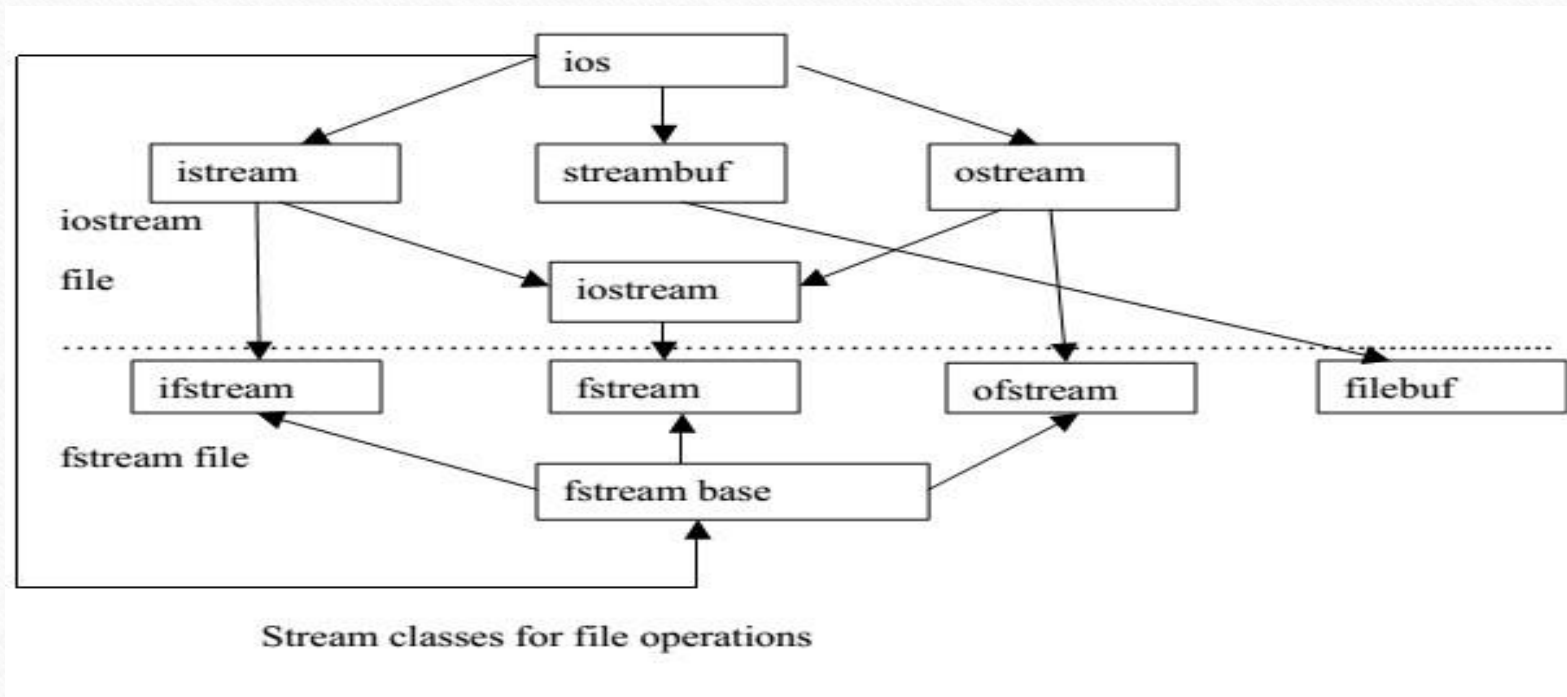
File Input and Output Stream



File Input and Output Stream

- The I/O system of C++ handles file operations in the same way as console input output stream.
- It uses **file stream as an interface** between program and the files.
- The stream that supplies data to the program is called as **input stream** and the one that receives the data from the program is called as **output stream**.
- In other words input stream extracts data from the file and output stream inserts data into the file.

Stream Classes for file operations



fstream.h

- `fstream.h` is the header file or library that provides set of classes for file handling methods to read and write from a file.
- It includes `ifstream`, `ofstream` and `fstream` classes which are derived from `fstreambase` class and corresponding `iostream.h` class.

Stream class Hierarchy

- There are 2 header files involved in this hierarchy
 - `iostream.h` for stream classes
 - `fstream.h` for file stream classes
- `ios` is the base class of stream class from which the base class of file stream class is derived which is `fstreambase`.
- For performing primitive operations there is a class called `streambuf` derived from `ios` class.

Stream class Hierarchy

- Similarly to perform the operations in file there is class named filebuf which is derived from streambuf class.
- We have 2 classes istream and ostream derived from ios
- iostream is derived from istream and ostream
- ifstream class is derived from fstreambase and istream
- ofstream class is derived from fstreambase and ostream
- fstream is derived from fstreambase and iostream class.

- **ios class** – This class is the base class for all stream classes. The streams can be input or output streams. This class defines members that are independent of how the templates of the class are defined.
- **istream Class** – The istream class handles the input stream in c++ programming language. These input stream objects are used to read and interpret the input as a sequence of characters. The cin handles the input.
- **ostream class** – The ostream class handles the output stream in c++ programming language. These output stream objects are used to write data as a sequence of characters on the screen. cout and puts handle the out streams in c++ programming language.

Class	Contents
filebuf	It sets the file buffer to read and write It has open() and close() as members
fstreambase	It provides operations common to file stream. Contains open() and close() functions.
ifstream	It provides input operation. It has open() with default input mode It inherits get(), getline(), read() seekg() and tellg() from istream
ofstream	It provides output operation. It has open() with default input mode It inherits pet(), getline(), read() seekp() and tellp() and write() from ostream
fstream	It provides support for input and output operations which are simultaneous

Operations on File

- Opening and Closing a file.
- Reading and Writing a File
- Detecting end of file
- Updation of file.

Opening and Closing a File

- When we are using the file we must think about the name of file, structure of the file and opening mode of the file.
- We can open a file using 2 ways:-
 - Opening a file using Constructor
 - Opening file using `open()` member function

Opening a file using Constructor

- In order to perform file operations the file should be open. As we know Constructor is used to initialize an object while it is being created.
- Here a Constructor is used to initialize the file name which is used with the file stream object.

Opening a file using Constructor

- We need to use the appropriate classes for this ie: either ofstream or ifstream.
- ifstream and ofstream are classes
- We need to create object of this class.

Syntax is:-

```
classname objectname(filename);
```

```
ifstream infile("emp.txt");
```

If file exists then it will perform the appropriate operation.

Opening a file using open()

A file must be opened before you perform reading or writing operation on the file.

We can make use of the objects to open the file.

open() is a member function of class fstream ifstream and ofstream

Standard Syntax of opening a file using open function is:-

```
void open (const char *filename, ios::openmode);
```

- Where **filename** is a string representing the name of the file to be opened, and **mode** is an optional parameter with a combination of the following flags.
- It requires 2 parameters where first parameter is name of the file and second parameter is the mode of opening file.

Modes of Opening Files

- `ios::in:-` Open for input operations.
- `ios::out:-` Open for output operations.
- `ios::binary:-` Open in binary mode.
- `ios::ate:-` Set the initial position at the end of the file.
If this flag is not set, the initial position is the beginning of the file.
- `ios::app:-` All output operations are performed at the end of the file, appending the content to the current content of the file.
- `ios::trunc` -If the file is opened for output operations and it already exists, its previous content is deleted and replaced by the new one.

Combine modes of operation

- We can combine the file opening modes.
- For example:- `fstream ifile;`

```
ifile.open("a.txt",ios::in || ios::out);
```

Closing a File

- When a C++ program terminates it automatically flushes all the streams, release all the allocated memory and close all the opened files. But it is always a good practice that a programmer should close all the opened files before program termination. `Close()` is a member function of `fstream`, `ifstream` and `ofstream`. `void close();`

- To close a file the syntax is

`objectname.close();`


```
#include <iostream.h>
```

```
int main ()
```

```
{
```

```
char ch;
```

```
ifstream ifile;
```

```
ifile.open ("a.txt", ios::in);
```

```
if (!ifile)
```

```
{
```

```
    cout<<"Error in opening file";
```

```
    exit(0);
```

```
}
```

Program cont...

```
while(!ifile.eof())  
{  
    ifile.get(ch);  
    cout<<ch;  
}  
ifile.close();  
}
```

Write a C++ program to write data to a text file.

```
#include<iostream.h>
```

```
#include<fstream.h>
```

```
void main()
```

```
{
```

```
char ch;
```

```
ofstream fout("a.txt");
```

```
cin>>ch;
```

```
while(ch!='#')
```

```
{
```

```
    fout<<ch;
```

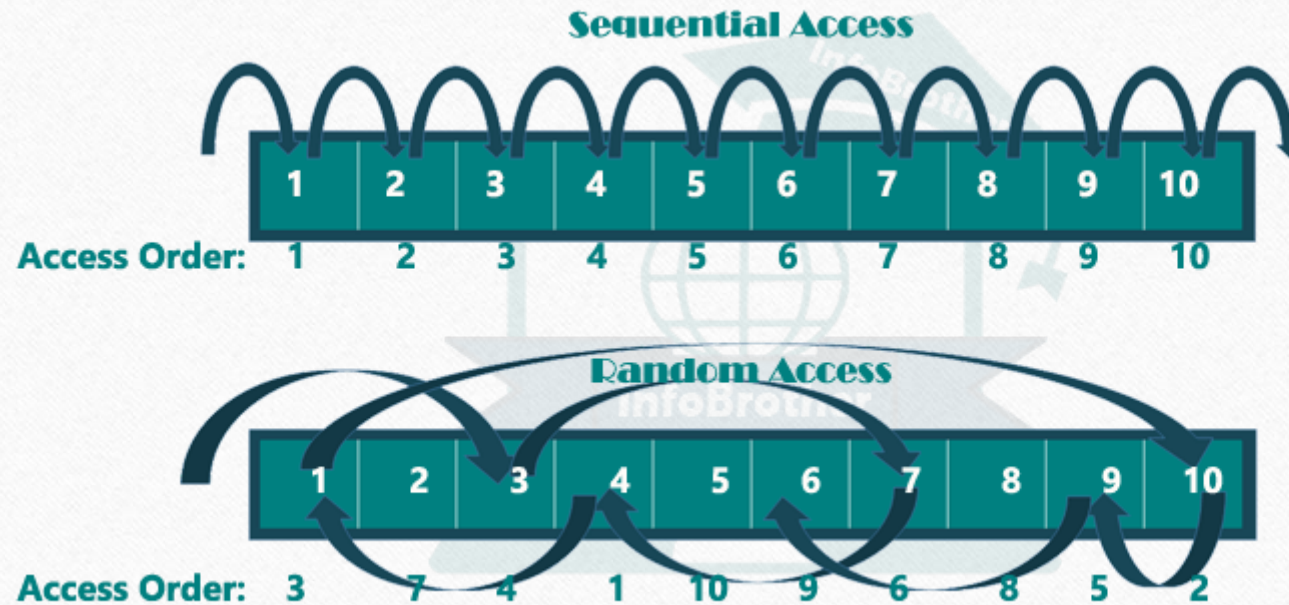
```
    cin>>ch;
```

```
}
```

```
fout.close();
```

```
}
```


Sequential Vs Random Access



Random Access of File

- Every file maintains two pointers called **get_pointer (in input mode file)** and **put_pointer (in output mode file)** which tells the current position in the file where reading or writing will take place. (A file pointer in this context is not like a C++ pointer but it works like a book-mark in a book.). These pointers help attain random access in file.
- That means moving directly to any location in the file instead of moving through it sequentially.

Random Access of File

- There may be situations where random access is the best choice.
- For example, if you have to modify a value in record no 21, then using random access techniques, you can place the file pointer at the beginning of record 21 and then straight-way process the record. If sequential access is used, then you'll have to unnecessarily go through first twenty records in order to reach at record 21.

Random Access of File

- In C++, random access is achieved by manipulating `seekg()`, `seekp()`, `tellg()` and `tellp()` functions.
- The `seekg()` and `tellg()` functions allow you to set and examine the `get_pointer`, and the `seekp()` and `tellp()` functions perform these operations on the `put_pointer`.
- The `seekg()` and `tellg()` functions are for input streams (`ifstream`) and
- `seekp()` and `tellp()` functions are for output streams (`ofstream`).

Seekg ()

- seekg() function is used to move/sets the get pointer at the desired/particular position to getting/reading data from a file (External file).
- If seekg() function is used to move/sets the get pointer at the particular location. So then by using tellg() function we easily get the current position of the get pointer from the external file.
- **Syntax:** seekg(offset, direction/seekdir);
- Using this function the **stream** pointer is changed the position/location i.e. counting pointer position from the beginning of the file.

Seekg ()

- **Stream** means the flow of data that acts as an interface between files and programs. Here, stream is input stream that is coming from the **ifstream** class.
- In this position/location of the get pointer is set to an offset value that is relative to some specific point that determined by the direction of the parameter. offsetvalue is an integer type. And direction of get pointer is of type **seekdir**, which determines the pointer position from where offset value is counted from.
- After moving the position of the get pointer, **read()** function is used to getting/reading the data from an external file.

Seekg ()

- **Offset/Seekdir values are:**
- `ios::beg`: beginning of a file
- `ios::end`: end of a file
- `ios::cur`: current position of a file

```
#include <fstream.h>
#include <iostream.h>
int main ()
{
    fstream obj;
    obj.open("test.txt", ios::in);
    int pos=6;
    obj.seekg(pos);
    while(!obj.eof())
    {
        char ch;
        obj>>ch;
        cout<<ch;
    }
    obj.close();
}
```

Seekp()

- Whereas seekp() function is used to move/sets the put pointer at the particular/desired location to putting the data into a file (External file).
- And If **seekp()** is used to move the put pointer at the particular/desired location. So then by using tellp() function we easily get the current position of the put pointer from the file.
- **Syntax:** seekp (offset, direction/seekdir);

Seekp()

- Using this function the **stream** pointer is changed the position/location i.e. counting pointer position from the beginning of the file.
- **Stream** means the flow of data that acts as an interface between files and programs. Here, stream is output stream that is coming from the **ofstream** class.
- Same as seekg() function, here also position/location of the put pointer is set to an offset value that is relative to some specific point that determined by the direction of the parameter. offsetvalue is an integer type. And direction of put pointer is of type **seekdir**, which determines the pointer position from where offset value is counted from.

Seekp()

- After moving the position of the put pointer, **write()** function is used to putting/writing the data into an external file.
- **Offset/Seekdir values are:**
 - ios::beg: beginning of a file
 - ios::end: end of a file
 - ios::cur: current position of a file

```
#include <iostream.h>
#include <fstream.h>
int main ()
{
    fstream obj;
    obj.open ("test.txt", ios::out);
    obj<<"Hello World";
    int pos = 6;
    obj.seekp(pos-1);
    obj<<"...And here the text changed";
    obj.close();
    return 0;
}
```


tellg()

- `tellg()` tells the current pointer position in the text file.
But instead of put pointer, it tells the get pointer's location
- Say we have entered 20 characters in a text file, and you want to read it.
But along with reading you also want to know the position of the last position in the text file.
That is where `tellg()` comes into play.
It would give you the output 19 since counting starts from 0.

```
#include <iostream.h>
#include <fstream.h>
using namespace std;
int main ()
{
    fstream obj;
    obj.open ("test.txt", ios::in);
    char ch;
    int pos;
    while(!obj.eof())
    {
        obj>>ch;
        pos = obj.tellg();
        cout<<pos<<". "<<ch<<"\n";
    }
    obj.close();
}
```

tellp ()

- `tellp()` tells the current pointer position in the text file.
- Say we have entered 20 characters in a text file. Your current pointer is at 19. Since pointer value starts from 0 therefore $20-1=19$. So if we use `tellp()` in this text file we would get an output which is 19 of type `int`.
- Using `tellp()` we can edit the text file from the last position in the text file or wherever the current pointer is.


```
#include <iostream.h>
```

```
#include <fstream.h>
```

```
int main ()
```

```
{
```

```
    fstream obj;
```

```
    obj.open ("test.txt", ios::out);
```

```
    obj<<"Hello World";
```

```
    int pos;
```

```
    pos = obj.tellp();
```

```
    cout<<pos;
```

```
    obj.close();
```

```
}
```

Operation on Binary File

- A binary file is a file which stores data in binary format.
- This file is not human readable
- To read a binary file in C++ use read method.
- It extracts a given number of bytes from the given stream and place them into the memory, pointed to by the first parameter. If any error is occurred during reading in the file, the stream is placed in an error state, all future read operation will be failed then.

Operation on Binary File.

- The functions specifically designed to read and write binary data sequentially are: write and read. The first one (write) is a member function of ostream (inherited by ofstream). And read is a member function of istream (inherited by ifstream). The write () and read () function reads and writes blocks of binary data. Syntax of read and write () is as follows: -
- `file.read ((char *) &obj, sizeof (obj));`
- `file.write ((char *) &obj, sizeof (obj));`

Write a C++ program to read and write data from a binary file.

```
#include <iostream.h>
#include <fstream.h>
class Employee
{
int empID;
char empName[100] ;
public :
```

```
void readEmployee ()
{
cout<<"EMPLOYEE DETAILS"<<endl;
cout<<"ENTER EMPLOYEE ID: " ;
cin>>empID;
cout<<"ENTER NAME OF THE EMPLOYEE : “
cin>>empName;
}
void displayEmployee()
{
cout<<"EMPLOYEE ID: "<<empID<<endl
<<"EMPLOYEE NAME: "<<empName<<endl;
}
};
```

```
int main ()
{
Employee emp;
emp.readEmployee ();
fstream file;
file.open ("emp.dat",ios:: out | ios::binary);
if (!file)
{
cout<<"Error in creating file...\n";
exit(0);
}
file.write ((char*)&emp,sizeof (emp));
file.close ();
cout<<"Date saved into file the file.\n";
```



```
file.open ("emp.dat", ios::in | ios:: binary);  
if(!file)  
{  
cout<<"Error in opening file...\n";  
exit (0);  
}  
if(file.read((char*)&emp,sizeof(emp)))  
{  
cout<<"Data from file..\n";  
emp.displayEmployee ();  
}  
file.close();  
return 0;  
}  
}
```

Exception Handling

- An exception is a problem that arises during the execution of a program.
- A C++ exception is a response to an exceptional circumstance that arises while a program is running, such as an attempt to divide by zero.
- Exceptions provide a way to transfer control from one part of a program to another.

Concept cont..

- C++ exception handling is built upon three keywords: **try**, **catch**, and **throw**.
- **throw** – A program throws an exception when a problem shows up. This is done using a **throw** keyword.
- **catch** – A program catches an exception with an exception handler at the place in a program where you want to handle the problem. The **catch** keyword indicates the catching of an exception.
- **try** – A **try** block identifies a block of code for which particular exceptions will be activated. It's followed by one or more catch blocks.

Concept cont..

- Assuming a block will raise an exception, a method catches an exception using a combination of the **try** and **catch** keywords. A try/catch block is placed around the code that might generate an exception. Code within a try/catch block is referred to as protected code, and the syntax for using try/catch as follows –

Types of bugs

1. Program might have bugs.
2. Bugs are of two types
 - - Logic error (occurs due to poor understanding of problem and solution problem)
 - - Syntactic error (arise due to poor understanding of the language itself)

-
- - Syntactic error (arise due to poor understanding of the language itself)

We often come across some peculiar problem other than logic and syntax error.

They are known as exception

Exception are run time anomalies or unusual conditions that a program may encounter while executing

Anomalies includes condition such as division by zero , access to an array outside of its bounds, running out of memory

Types of Exception

- Exception are of two types
 - synchronous exception (errors such as “out-of-range index” and “overflow”)
 - Asynchronous exception (errors caused by events beyond the control of the program such as keyboard interrupts)

Tasks performed by error handling code

1. Find the problem
(Hit the exception)
2. Inform that an error has occurred
(Throw the exception)
3. Receive the error information
(Catch the exception)
4. Take corrective actions
(Handle the exception)

3 keywords used

1. Try

keyword try is used to preface a block of statement(surrounded by braces) which may generate exception .This block of statement is called try block.

2. Throw

when an exception is detected it is thrown using throw statements in try block

`throw (exception);`

`throw exception;`

`throw; //`used for rethrowing an exception

3 keywords used

3 .Catch

It is defined by keyword catch. It catches the exception thrown by throw statement in the try block ,and handles it appropriately

```
.....  
try  
{  
    .....  
    throw exception          //block of statements which  
    .....                  // detects and throws an exception  
}  
catch (type arg)            //catches exception  
{  
    .....                  //block of statements that  
    .....                  // handles the exception  
    .....  
}  
.....
```

```
#include<iostream.h>
main()
{
    int a , i;
    cout<<"\nEnter the value of a :";
    cin >> a;
    try
    {
        if(a>0)
        {
            cout << "\n No. is positive:" << a ;
        }
        else
        {
            throw (i);
        }
    }
    catch(int i)
    {
        cout << "The no. is negative exception";
    }
}
```